WEEK 7 & 8: SQL FUNDAMENTALS

Interacting with Databases

TRANSITIONING TO DATABASE INTERACTION

Moving from design theory to hands-on SQL execution. execution.

- **Goal**: Enable students to begin interacting with live databases through standard SQL commands.
- **Practical Focus**: Shift from conceptual modeling to technical implementation and data manipulation.
- **Core Skills**: Mastering the SQL environment and the the fundamental CRUD cycle.
- **Outcome**: Students will build a functional Student Student Record or Inventory System as their final final project.

SETTING UP THE SQL ENVIRONMENT

ENVIRONMENT OVERVIEW

- Understanding the role of the DBMS (e.g., MySQL, PostgreSQL).
- Familiarization with SQL client interfaces and terminals.
- Distinction between DDL (Definition) and DML (Manipulation).

FIRST COMMAND

```
CREATE DATABASE school_db;
```

BEST PRACTICES

- Always verify database creation before building tables. tables.
- Use consistent naming conventions for databases.
- Master the environment before moving to complex queries.

DEFINING STRUCTURE: CREATE, ALTER, & DROP

DDL FUNDAMENTALS

- **CREATE TABLE**: Defining entities, attributes, and constraints.
- **ALTER**: Modifying existing structures without data loss. loss.
- **DROP**: Safely removing tables or databases.
- **Data Types**: Choosing between INT, VARCHAR, and DATE.

IMPLEMENTATION EXAMPLE

```
CREATE TABLE Students(
StudentID INT PRIMARY KEY,
Name VARCHAR(50)
);
```

KEY CONSIDERATIONS

- Always define a **Primary Key** for uniqueness.
- Use **VARCHAR(n)** for variable-length strings.
- Ensure structural changes are documented.

POPULATING THE DATABASE: INSERT

Bringing your tables to life with initial data entries.

- Transition from schema definition to data population. population.
- Ensuring data aligns with defined constraints.

SQL SYNTAX

```
INSERT INTO Students
VALUES (1, 'John');
```

INTEGRITY CHECKS

- **PK Constraints**: Prevents duplicate records.
- **Bulk Loading**: Efficient multi-row insertion.
- **Null Handling**: Managing missing data points.

RETRIEVING INFORMATION: THE POWER OF SELECT

The most frequent operation in database management.

Key Insight:

Data is only as useful as your ability to retrieve and filter it efficiently.

SYNTAX & TECHNIQUES

```
SELECT * FROM Students;
```

- **Basic Query**: Use the asterisk (*) to view all records in a table.
- **Filtering**: Use the WHERE clause to isolate specific data points.
- **Projection**: Select specific columns to improve performance and clarity.
- **Sorting**: Use ORDER BY to organize results numerically or alphabetically.

MAINTAINING DATA: UPDATE AND DELETE

LIFECYCLE MANAGEMENT

- **UPDATE**: Modifying existing values within a table.
- **DELETE**: Removing specific records that are no longer valid.
- **The WHERE Clause**: The critical filter that targets specific rows.

SQL SYNTAX

```
UPDATE Students
SET Name = 'Juan'
WHERE StudentID = 1;
```

```
DELETE FROM Students
WHERE StudentID = 1;
```

SAFETY FIRST:

Always run a **SELECT** query with your **WHERE** criteria before executing an executing an UPDATE or DELETE to verify the target rows.

LAB PROJECT: BUILDING A FUNCTIONAL SYSTEM

PROJECT OPTIONS

- **Student Record System**: Manage students, courses, and grades.
- **Inventory System**: Track products, suppliers, and orders. orders.

CORE REQUIREMENTS

- **CRUD Lifecycle**: Implement Create, Read, Update, and and Delete for all entities.
- **Relational Integrity**: Use at least 3 related tables with PK/FK enforcement.
- **Data Validation**: Ensure constraints prevent invalid data entry.

Final Deliverable:

A complete SQL script containing schema creation, sample data insertion, insertion, and demonstration queries.

SUMMARY & SUCCESS CRITERIA

Key milestones for mastering SQL Fundamentals.

MILESTONES FOR MASTERY

- **Syntax Precision**: Mastery of standard SQL keywords and proper statement termination.
- **Logical Flow**: Understanding the sequence from Database → Table → Data.
- **Error Handling**: Ability to interpret DBMS error messages and debug SQL scripts.
- **Final Goal**: Successfully delivering a project that demonstrates full CRUD capability.

Ready for Interaction:

Students are now equipped to move from theoretical design to practical practical database management.